ASSIGNMENT-12.5

B.HARSHAVARDHAN

2303A52T02

Q: Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Lab Objectives:

• Apply AI-assisted programming to implement and optimize

Week6 -

Friday

sorting and searching algorithms.

• Compare different algorithms in terms of efficiency and use cases.

• Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The

system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python.

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

Student Task

• Use AI to suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Student Task

• Recommend searching and sorting algorithms using AI.

• Justify the chosen algorithms.

• Implement Python functions.

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop

name, soil moisture level, temperature, and yield estimate. Farmers

need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

Student Task

• Use AI-assisted reasoning to select algorithms.

• Justify algorithm suitability.

• Implement searching and sorting in Python.

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID,

airline name, departure time, arrival time, and status. The system

must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

• Use AI to recommend algorithms.

• Justify the algorithm selection.

• Implement searching and sorting logic in Python.

CODE: # Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task 1: Merge Sort Implementation

```
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Time Complexity:
```

```
    Best, Average, Worst: O(n log n)
  Space Complexity:
    O(n) - due to auxiliary arrays used during merging.
  """
  if len(arr) <= 1:
    return arr
  mid = len(arr) // 2
  left = merge_sort(arr[:mid])
  right = merge_sort(arr[mid:])
  return merge(left, right)

def merge(left, right):
  result = []
  i = j = 0
  while i < len(left) and j < len(right):
    if left[i] < right[j]:
      result.append(left[i])
      i += 1
    else:
      result.append(right[j])
      j += 1
  result.extend(left[i:])
  result.extend(right[j:])
  return result

# Test cases for merge_sort
print("Merge Sort Test Cases:")
print(merge_sort([5, 2, 9, 1, 5, 6]))  # [1, 2, 5, 5, 6, 9]
```

```
print(merge_sort([]))              # []
print(merge_sort([1]))             # [1]
print(merge_sort([3, 2, 1]))       # [1, 2, 3]

# Task 2: Binary Search Implementation

def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the target element.

    Args:
        arr (list): Sorted list of elements.
        target: Element to search for.

    Returns:
        int: Index of target if found, else -1.

    Time Complexity:
        Best Case: O(1) (if the middle element is the target)
        Average/Worst Case: O(log n)
    Space Complexity:
        O(1) - iterative implementation.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
```

```
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Test cases for binary_search
print("\nBinary Search Test Cases:")
sorted_arr = [1, 2, 3, 4, 5, 6, 7]
print(binary_search(sorted_arr, 4))  # 3
print(binary_search(sorted_arr, 1))  # 0
print(binary_search(sorted_arr, 7))  # 6
print(binary_search(sorted_arr, 8))  # -1

# Task 3: Smart Healthcare Appointment Scheduling System

# Recommendation:
# - For searching by appointment ID: Binary Search (if sorted), else Hash Map for O(1) lookup.
# - For sorting by time or fee: Merge Sort (stable, O(n log n)).

# Implementation:
class Appointment:
    def __init__(self, appt_id, patient, doctor, time, fee):
        self.appt_id = appt_id
        self.patient = patient
        self.doctor = doctor
        self.time = time
```

```
        self.fee = fee


def search_appointment_by_id(appointments, appt_id):
    """
    Linear search for appointment by ID.
    """
    for idx, appt in enumerate(appointments):
        if appt.appt_id == appt_id:
            return idx
    return -1


def sort_appointments_by_time(appointments):
    """
    Sort appointments by time using Merge Sort.
    """
    if len(appointments) <= 1:
        return appointments
    mid = len(appointments) // 2
    left = sort_appointments_by_time(appointments[:mid])
    right = sort_appointments_by_time(appointments[mid:])
    return merge_appointments_by_time(left, right)


def merge_appointments_by_time(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].time < right[j].time:
            result.append(left[i])
```

```
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def sort_appointments_by_fee(appointments):
    return sorted(appointments, key=lambda x: x.fee)

# Task 4: Railway Ticket Reservation System

# Recommendation:
# - Search by ticket ID: Binary Search (if sorted), else Hash Map.
# - Sort by travel date or seat number: Merge Sort.

class Ticket:
    def __init__(self, ticket_id, passenger, train_no, seat_no, travel_date):
        self.ticket_id = ticket_id
        self.passenger = passenger
        self.train_no = train_no
        self.seat_no = seat_no
        self.travel_date = travel_date

def search_ticket_by_id(tickets, ticket_id):
    for idx, ticket in enumerate(tickets):
        if ticket.ticket_id == ticket_id:
```

```
            return idx
    return -1

def sort_tickets_by_date(tickets):
    return sorted(tickets, key=lambda x: x.travel_date)

def sort_tickets_by_seat(tickets):
    return sorted(tickets, key=lambda x: x.seat_no)

# Task 5: Smart Hostel Room Allocation System

# Recommendation:
# - Search by student ID: Hash Map or Linear Search.
# - Sort by room number or allocation date: Merge Sort.

class Allocation:
    def __init__(self, student_id, room_no, floor, alloc_date):
        self.student_id = student_id
        self.room_no = room_no
        self.floor = floor
        self.alloc_date = alloc_date

def search_allocation_by_student_id(allocations, student_id):
    for idx, alloc in enumerate(allocations):
        if alloc.student_id == student_id:
            return idx
    return -1
```

```
def sort_allocations_by_room(allocations):
    return sorted(allocations, key=lambda x: x.room_no)

def sort_allocations_by_date(allocations):
    return sorted(allocations, key=lambda x: x.alloc_date)

# Task 6: Online Movie Streaming Platform

# Recommendation:
# - Search by movie ID: Hash Map or Binary Search.
# - Sort by rating or release year: Merge Sort.

class Movie:
    def __init__(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
        self.title = title
        self.genre = genre
        self.rating = rating
        self.release_year = release_year

def search_movie_by_id(movies, movie_id):
    for idx, movie in enumerate(movies):
        if movie.movie_id == movie_id:
            return idx
    return -1

def sort_movies_by_rating(movies):
    return sorted(movies, key=lambda x: x.rating, reverse=True)
```

```
def sort_movies_by_release_year(movies):
    return sorted(movies, key=lambda x: x.release_year, reverse=True)

# Task 7: Smart Agriculture Crop Monitoring System

# Recommendation:
# - Search by crop ID: Hash Map or Linear Search.
# - Sort by moisture level or yield estimate: Merge Sort.

class Crop:
    def __init__(self, crop_id, name, moisture, temp, yield_est):
        self.crop_id = crop_id
        self.name = name
        self.moisture = moisture
        self.temp = temp
        self.yield_est = yield_est

def search_crop_by_id(crops, crop_id):
    for idx, crop in enumerate(crops):
        if crop.crop_id == crop_id:
            return idx
    return -1

def sort_crops_by_moisture(crops):
    return sorted(crops, key=lambda x: x.moisture)

def sort_crops_by_yield(crops):
```

```
    return sorted(crops, key=lambda x: x.yield_est, reverse=True)

# Task 8: Airport Flight Management System

# Recommendation:
# - Search by flight ID: Hash Map or Binary Search.
# - Sort by departure or arrival time: Merge Sort.

class Flight:
    def __init__(self, flight_id, airline, dep_time, arr_time, status):
        self.flight_id = flight_id
        self.airline = airline
        self.dep_time = dep_time
        self.arr_time = arr_time
        self.status = status

def search_flight_by_id(flights, flight_id):
    for idx, flight in enumerate(flights):
        if flight.flight_id == flight_id:
            return idx
    return -1

def sort_flights_by_departure(flights):
    return sorted(flights, key=lambda x: x.dep_time)

def sort_flights_by_arrival(flights):
    return sorted(flights, key=lambda x: x.arr_time)
```

OUTPUT:

```
PS C:\Users\HARSHAVARDHAN\OneDrive\Desktop\AI-CODING> & C:/Users/HARSHAVARDHAN/AppData/Local/Programs
e/Desktop/AI-CODING/ass-12.5.py
Merge Sort Test Cases:
[1, 2, 5, 5, 6, 9]
[]
[1]
[1, 2, 3]

Binary Search Test Cases:
3
0
6
-1
PS C:\Users\HARSHAVARDHAN\OneDrive\Desktop\AI-CODING>
```